

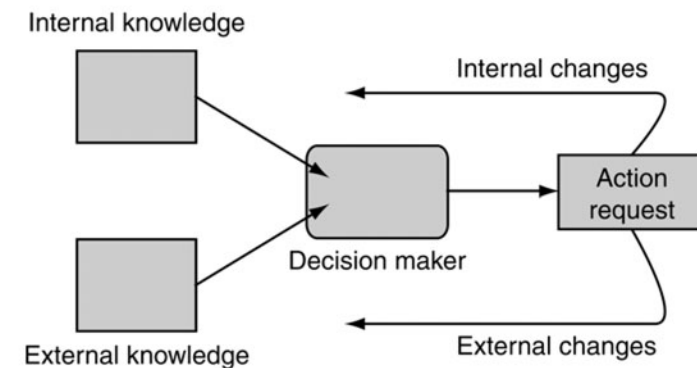
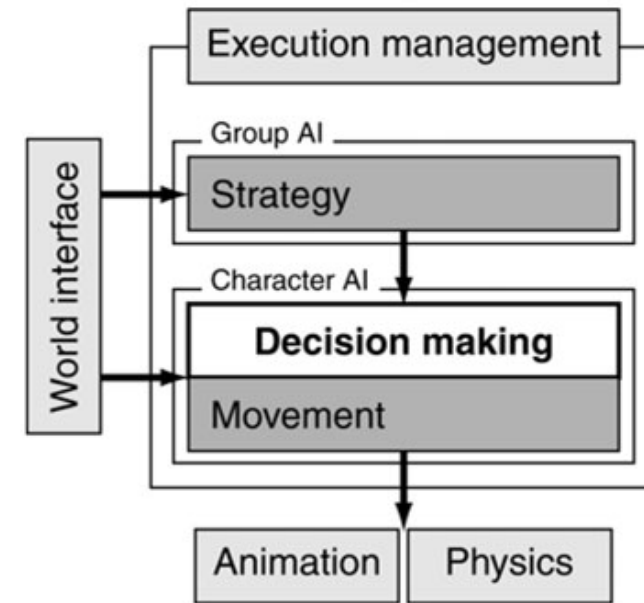
Video Game Programming: Decision Making AI

L13

Prof. Fabio Buttussi, PhD
Human-Computer Interaction Laboratory
Department of Mathematics, Computer Science, and Physics
University of Udine - Italy

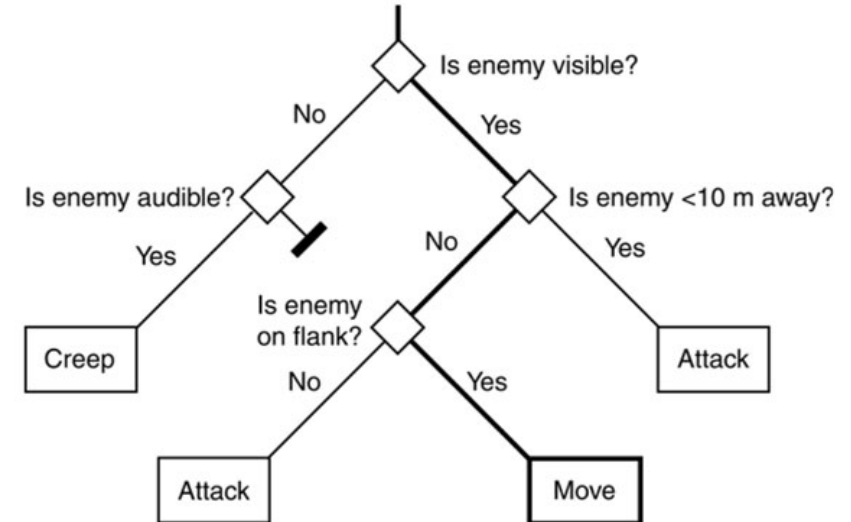
Decision making

- It is the middle component in the model in fig, but most techniques can be used also for strategy
- The input is the knowledge that a character possesses
- The output is an action request
- External knowledge about environment is the same for different techniques
- Internal knowledge about state or thought process varies



Decision trees

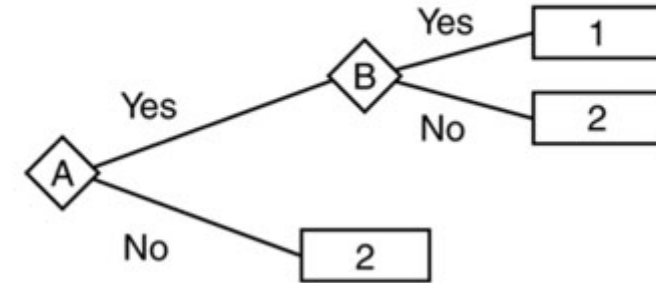
- Given a set of knowledge and actions, an NPC AI needs to request an appropriate action
- Decision trees do it in a modular and easy way
- They are made up of connected nodes for decision points
- For each node, starting from root, one of a set of options is chosen
- Choices lead either to further decisions or a final action (leaf)



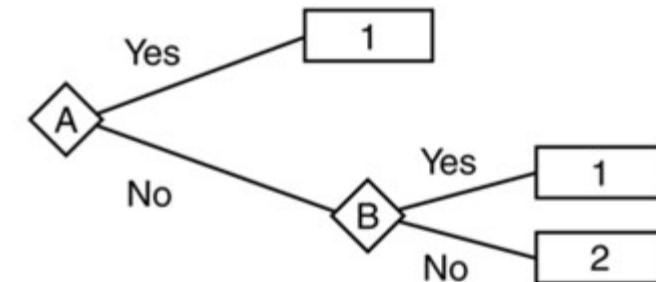
Decision types

- Nodes typically check a single value without Boolean logic
- To AND or OR two decisions together, they are placed in series like in fig.
- While decisions can have more options and branches, the most common are binary decision trees
- They are often coded to directly access game state
- Balanced trees are optimal to making fast decisions

If A AND B then action 1, otherwise action 2

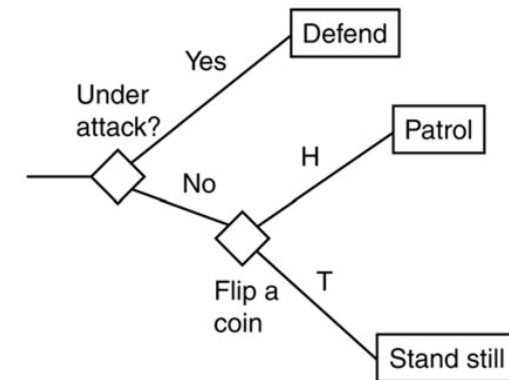


If A OR B then action 1, otherwise action 2



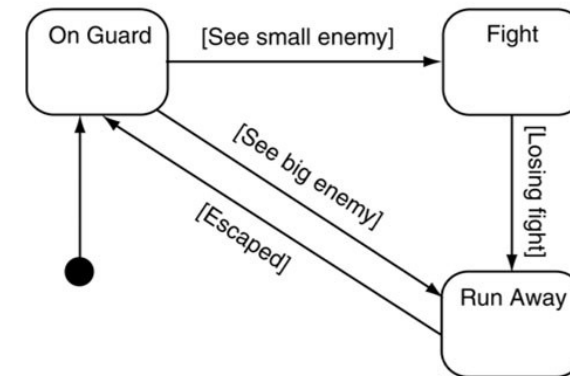
Adding randomness

- To avoid too predictable behaviors, randomness can be added as in fig.
- To avoid continuous changes due to randomness
 - When the decision is first considered, a random choice is taken and stored
 - At next decision time, the previous choice is automatically taken, at least for a time span



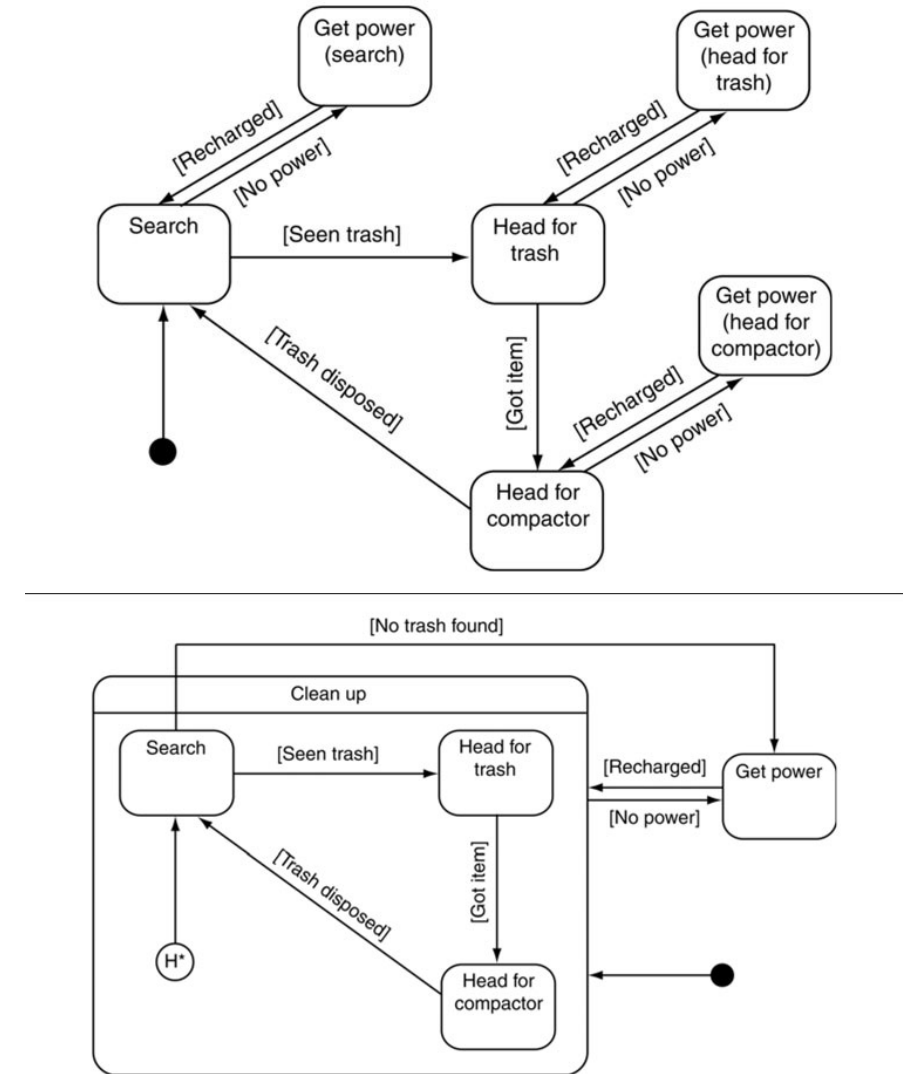
Finite state machines

- Each NPC has a current state
- As long as NPC remains in a state, it will do the same action
- When conditions are met (trigger), FSM can follow a transition to a new state (fire)
- While in decision trees the same set of decisions is always used, in FSMs, only transitions from current state are considered



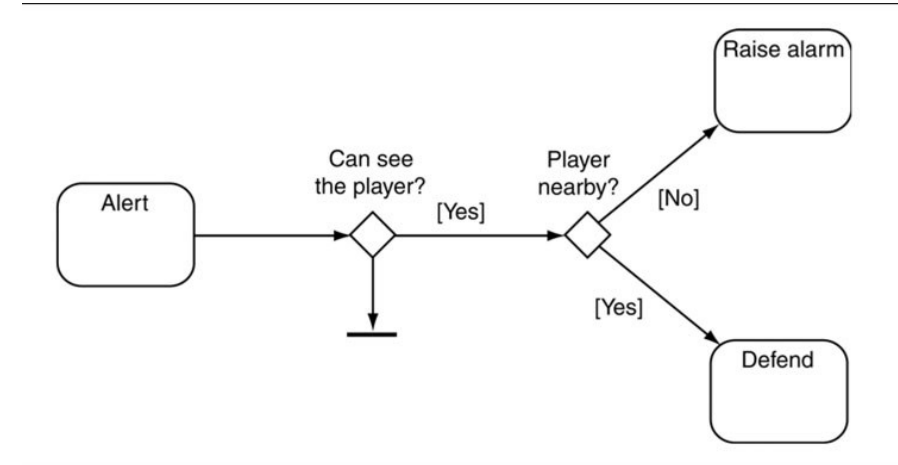
Hierarchical SMs

- With FSMs is difficult to model alarm / interrupting behaviors
- This can be solved with hierarchical state machines
- Transitions can involve states, sub-states or entire FSMs
- N.B.: Support of different transitions types varies with implementation



FSMs with decision trees

- Decision trees have problems due to the lack of state
- FSM can have complex transitions
- We can combine both techniques by replacing transitions with decision trees as in fig.

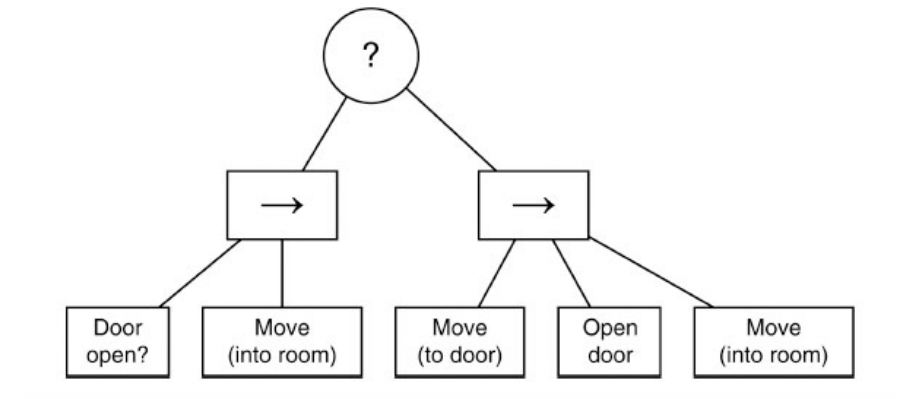


Behavior trees

- After Halo 2, behavior trees have become popular in games
- They are easy to understand and create also for non-programmers
- The main building block of a behavior tree is a **task**
- Tasks are decomposed into sub-trees to represent complex actions
- Tasks return with either success or failure
- In simple behavior trees, tasks can be:
 - Actions, which alter the state at leaf nodes
 - Conditions, which test properties at leaf nodes
 - Composites, which handle branches at internal nodes

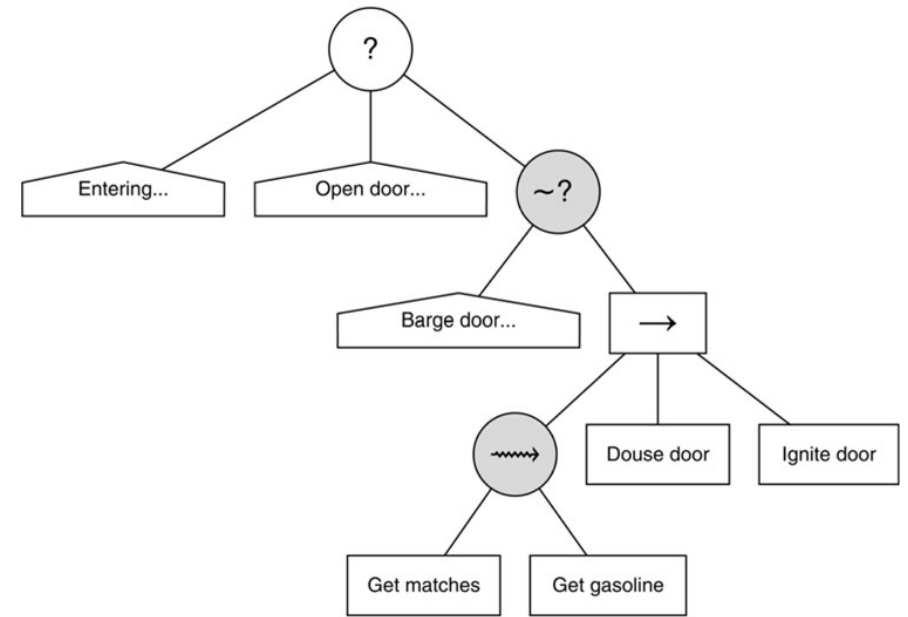
Selector and sequence

- Basic composite tasks are Selector and Sequence
- Selector will
 - return success on a child success
 - if a child fails, it tries another
 - if no child has success, it fails
- Sequence will
 - return fail on a child failure
 - if a child succeeds, it keeps going
 - if all children succeed, it succeeds



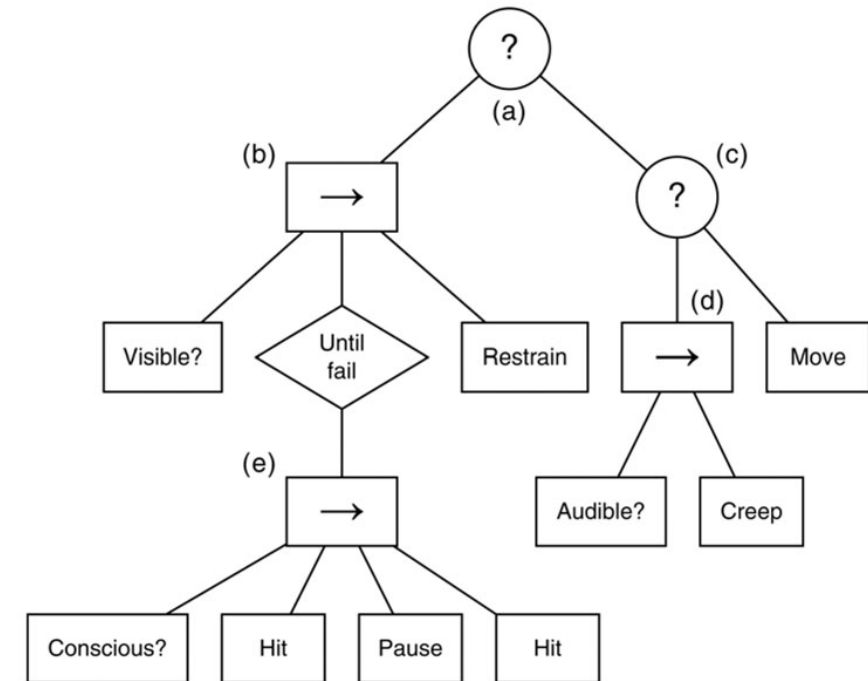
Reactive planning

- Behavior trees implement **reactive planning** to try alternatives in case of failure
- Alternatives are predefined, but can lead to believable NPC behaviors
- **Depth-first search** is used to visit
- **Partial order** constraints to try alternatives or sequences in a shuffled order can be added to reduce predictability (in fig.)



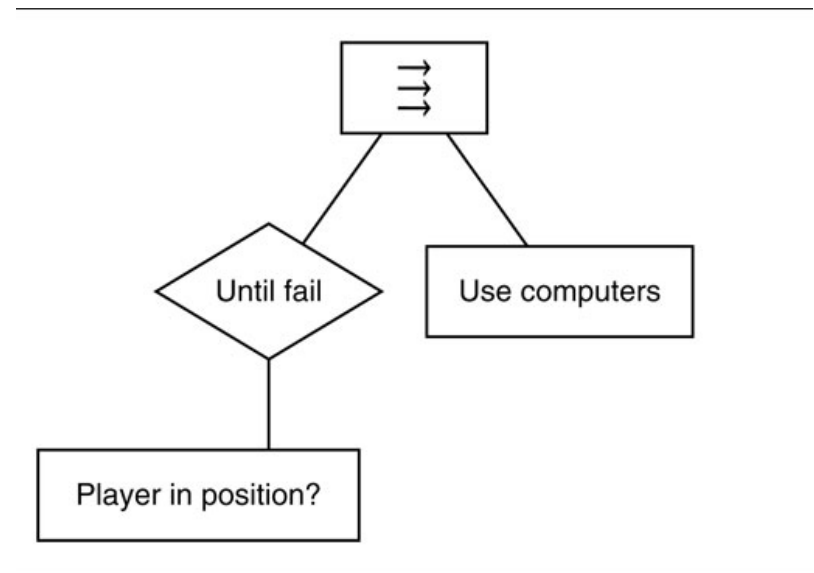
Decorators

- Decorators are an additional task type with only one child
- They can alter child behavior in different ways:
 - Limiting the number of times it can run
 - Keeping running it until it fails
 - Inverting return status
 - Guarding resources with semaphores



Parallel tasks

- Parallel is an additional type of composite task
- Its return status is like sequence, but children run simultaneously
- Failure in a child stops the parallel task, so all tasks need a method to force stopping after cleaning resources
- Parallel can be used to keep track of conditions while doing actions

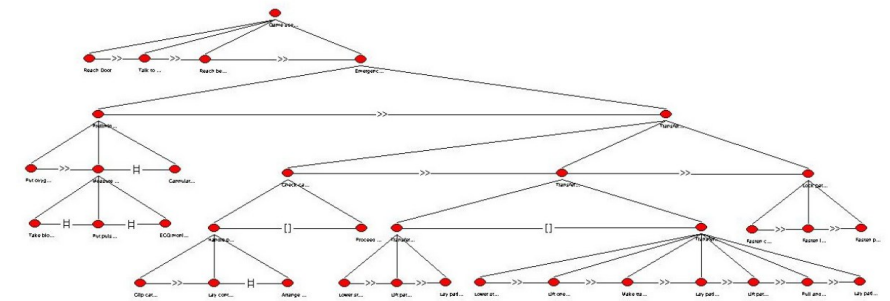


Adding data to behavior trees

- In real game implementation, tasks need to know what to work on and to share data with other tasks
- If the behavior tree is built dynamically (not often), tasks can be parameterized
- A commonly used approach is to separate tasks from data, putting the last on a blackboard
- Behavior trees or sub-trees can be reused for different characters

Concurrent Task Trees

- A task modeling formalisms with a wide set of constructs was introduced in Paternò, Mancini, Meniconi, ConcurTaskTrees: A Diagrammatic Notation for Specifying Task Models, Procs. IFIP International Conference on Human-Computer Interaction, 1997, pp. 362-369
- CTT were applied to serious games in Cabas Vidani, Chittaro, Using a Task Modeling Formalism in the Design of Serious Games for Emergency Medical Procedures, Conf. Games and Virtual Worlds for Serious Apps., 2009, IEEE, pp. 95-102



A medical procedure in CTT for a serious game

Interesting resources

- Isla, Handling complexity in the Halo 2 AI, Game Developers Conference, 2005
- Champandard, Dunstan, The Behavior Tree Starter Kit, <http://github.com/aigamedev/btsk>, 2012
- Champandard, Dunstan, The Behavior Tree Starter Kit, https://www.gameapro.com/GameAIPro/GameAIPro_Chapter06_The_Behavior_Tree_Starter_Kit.pdf, Game AI Pro, 2013
- Miyake et al., A Character Decision-Making System for FINAL FANTASY XV by Combining Behavior Trees and State Machines, http://www.gameapro.com/GameAIPro3/GameAIPro3_Chapter11_A_Character_Decision-Making_System_for_FINAL_FANTASY_XV_by_Combining_Behavior_Trees_and_State_Machines.pdf, Game AI Pro 3, 2017

Bibliografy

- Millington, AI for Games, 3rd ed., CRC Press, 2019
- Rabin, Game AI Pro 360: Guide to Architecture, CRC Press, 2020
- Rabin, Game AI Pro 360: Guide to Character Behavior, CRC Press, 2020
- Roberts, Artificial Intelligence in Games, CRC Press, 2023

